Taller: Polígonos sobre un mapa con geolocalización

Nombre: María Paz Puerta Acevedo

Datos generales y objetivos

Al final del taller cada estudiante tendrá una página web que lo ubica en un mapa, le permite dibujar un polígono con clics, calcula su área y perímetro, y lo exporta como GeoJSON.

Campo	Detalle
Asignatura	Programación Frontend / Páginas Web Dinámicas
Duración	4 horas (2 sesiones de 2 h)
Modalidad	Individual o en parejas
Herramientas	Navegador moderno, VS Code con Live Server, Leaflet 1.9.4, Turf.js 7, mapas de OpenStreetMap
Requisitos previos	DOM, eventos, funciones, arrays y objetos en JavaScript

Objetivos de aprendizaje. El estudiante será capaz de:

1. Obtener la ubicación del usuario con la API `navigator.geolocation` y manejar sus errores.
2. Renderizar un mapa interactivo con Leaflet y centrarlo en una coordenada.
3. Capturar eventos de clic sobre el mapa y almacenar coordenadas en un array.
4. Dibujar, editar y borrar un polígono a partir de ese array.
5. Calcular área y perímetro y exportar el resultado en formato GeoJSON.

Conceptos previos

Antes de programar, repase con el grupo estos cinco conceptos (20 minutos).

- **Coordenadas geográficas.** Un punto se describe con latitud (-90 a 90, norte/sur) y longitud (-180 a 180, este/oeste). Manizales está aproximadamente en lat 5.07, lng -75.52.
- **API de Geolocalización.** `navigator.geolocation.getCurrentPosition(exito, error, opciones)` pide permiso al usuario y devuelve `coords.latitude`, `coords.longitude` y `coords.accuracy` (precisión en metros). Solo funciona en contexto seguro: HTTPS o `localhost`. Por eso se usa Live Server y no se abre el archivo con doble clic.
- **Leaflet.** Librería JavaScript ligera para mapas. Sus piezas básicas son `L.map`, `L.tileLayer` (las imágenes del mapa), `L.marker`, `L.circle` y `L.polygon`.
- **Polígono.** Figura cerrada definida por un array ordenado de al menos 3 vértices `[lat, lng]`. Leaflet cierra la figura automáticamente uniendo el último punto con el primero.
- **GeoJSON.** Formato estándar para datos geográficos. Ojo: GeoJSON guarda las coordenadas como `[lng, lat]`, al revés que Leaflet, y el primer punto se repite al final para cerrar el anillo.

Desarrollo paso a paso

El proyecto se construye en 7 pasos sobre tres archivos; cada paso agrega código al final de `app.js` y se prueba en el navegador antes de seguir.

Paso 1. Estructura del proyecto (15 min)

Cree una carpeta `taller-poligono` con tres archivos: `index.html`, `estilos.css` y `app.js`. Ábrala en VS Code y lance Live Server.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Polígono con geolocalización</title>
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css">
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <header>
    <h1>Mi polígono</h1>
    <div class="controles">
      <button id="btnUbicar">Mi ubicación</button>
      <button id="btnCerrar" disabled>Cerrar polígono</button>
      <button id="btnDeshacer" disabled>Deshacer punto</button>
      <button id="btnLimpiar">Limpiar</button>
      <button id="btnExportar" disabled>Exportar GeoJSON</button>
    </div>
    <p id="info">Haz clic en el mapa para agregar vértices.</p>
  </header>
  <div id="mapa"></div>

  <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/@turf/turf@7/turf.min.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

```
body { margin: 0; font-family: system-ui, sans-serif; display: flex; flex-direction:
column; height: 100vh; }
header { padding: 10px 16px; background: #1e3a5f; color: white; }
h1 { margin: 0 0 8px; font-size: 1.3rem; }
.controles { display: flex; flex-wrap: wrap; gap: 8px; }
button { padding: 6px 12px; border: none; border-radius: 4px; cursor: pointer; }
button:disabled { opacity: .5; cursor: not-allowed; }
#info { margin: 8px 0 0; font-size: .9rem; }
#mapa { flex: 1; }
```

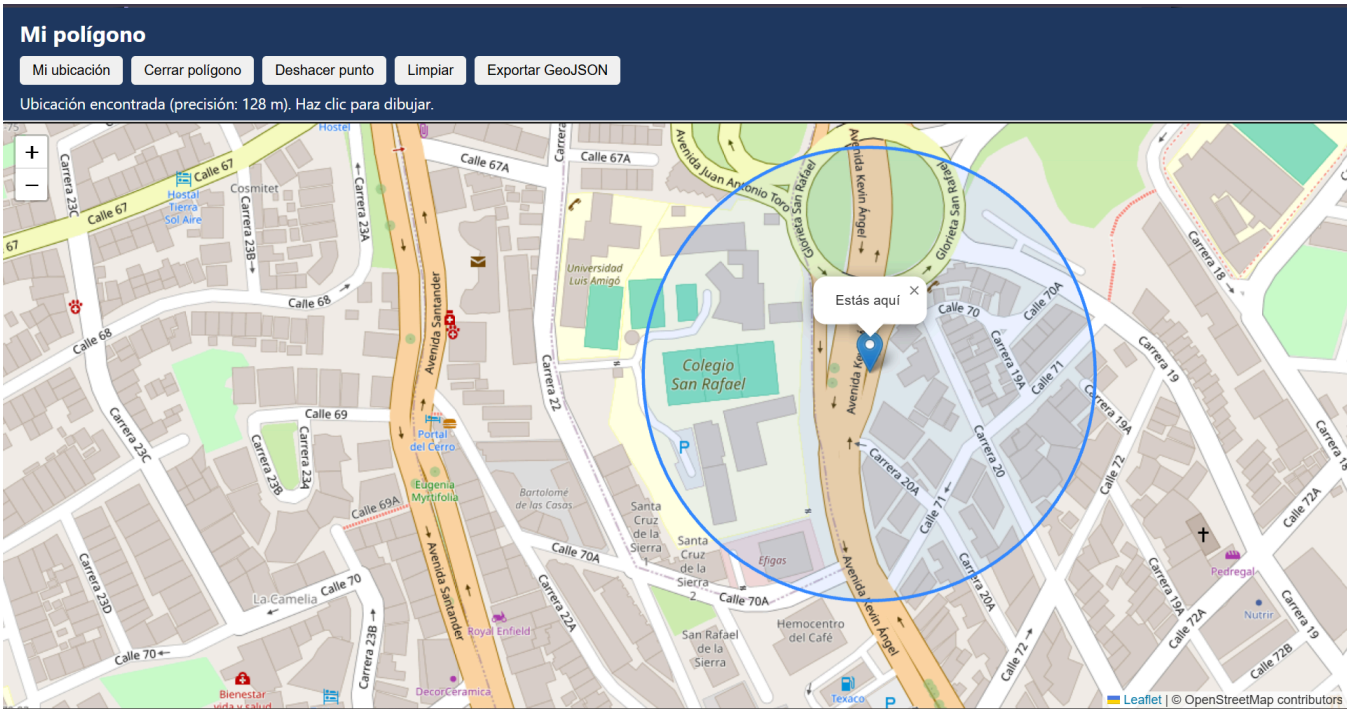
Pregunta de control:
¿Por qué el `<div id="mapa">` necesita una altura? (Sin altura, Leaflet dibuja un mapa de 0 px).

Respuesta:
Porque sin altura, Leaflet dibuja un mapa de 0 px, es decir, si el contenedor no tiene definida una altura, se dibujará un mapa de 0 px, el cual no se verá.

Paso 2. Mostrar el mapa (15 min)

```
const CENTRO_INICIAL = [5.0689, -75.5174]; // Manizales, por si falla la ubicación
const mapa = L.map('mapa').setView(CENTRO_INICIAL, 15);
L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {
maxZoom: 19,
attribution: '&copy; OpenStreetMap contributors'
}).addTo(mapa);
```

Verificación: se ve el mapa y se puede hacer zoom y arrastrar.



Paso 3. Geolocalización (25 min)

```
let marcadorUsuario = null;
let circuloPrecision = null;

function mostrarInfo(texto) {
  document.getElementById('info').textContent = texto;
}

function ubicarUsuario() {
if (!('geolocation' in navigator)) {
mostrarInfo('Tu navegador no soporta geolocalización.'); return;
  }
mostrarInfo('Buscando tu ubicación...');

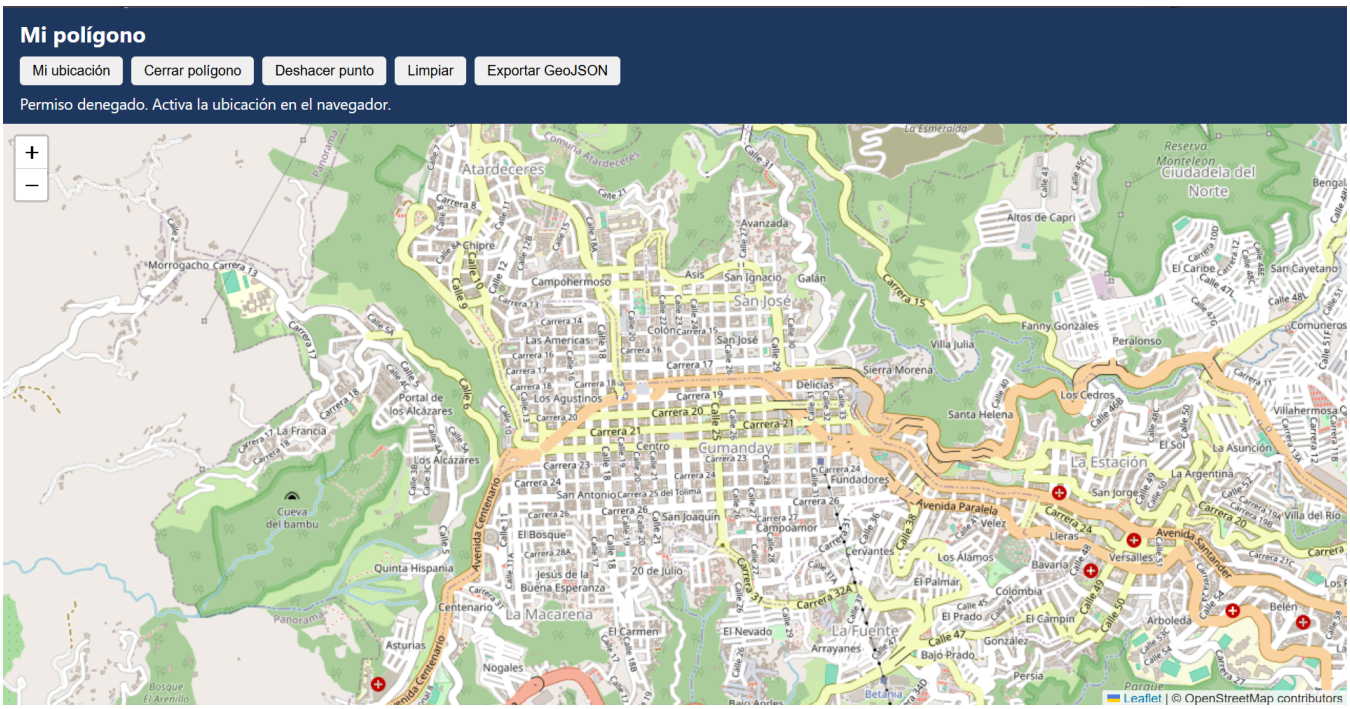
  navigator.geolocation.getCurrentPosition(
    (pos) => {
const { latitude, longitude, accuracy } = pos.coords; const
      punto = [latitude, longitude];

if (marcadorUsuario) mapa.removeLayer(marcadorUsuario); if
      (circuloPrecision) mapa.removeLayer(circuloPrecision);

      marcadorUsuario = L.marker(punto).addTo(mapa).bindPopup('Estás
aquí').openPopup();
```

```
document.getElementById('btnUbicar').addEventListener('click', ubicarUsuario);
ubicarUsuario();
```

Verificación: al aceptar el permiso aparece un marcador y un círculo azul. Pida a un estudiante que deniegue el permiso para ver el mensaje de error. Discuta por qué en un portátil la precisión puede ser de cientos de metros (se calcula por Wi-Fi o IP, no por GPS).



Paso 4. Capturar vértices con clics (25 min)

```
let puntos = [];    // array de [lat, lng]
let vertices = []; // marcadores visibles de cada vértice
let poligono = null;
const lineaGuia = L.polyline([], { color: '#e67e22', dashArray: '6 6' }).addTo(mapa);

mapa.on('click', (e) => {
if (poligono) return; // si ya está cerrado, no se agregan más puntos

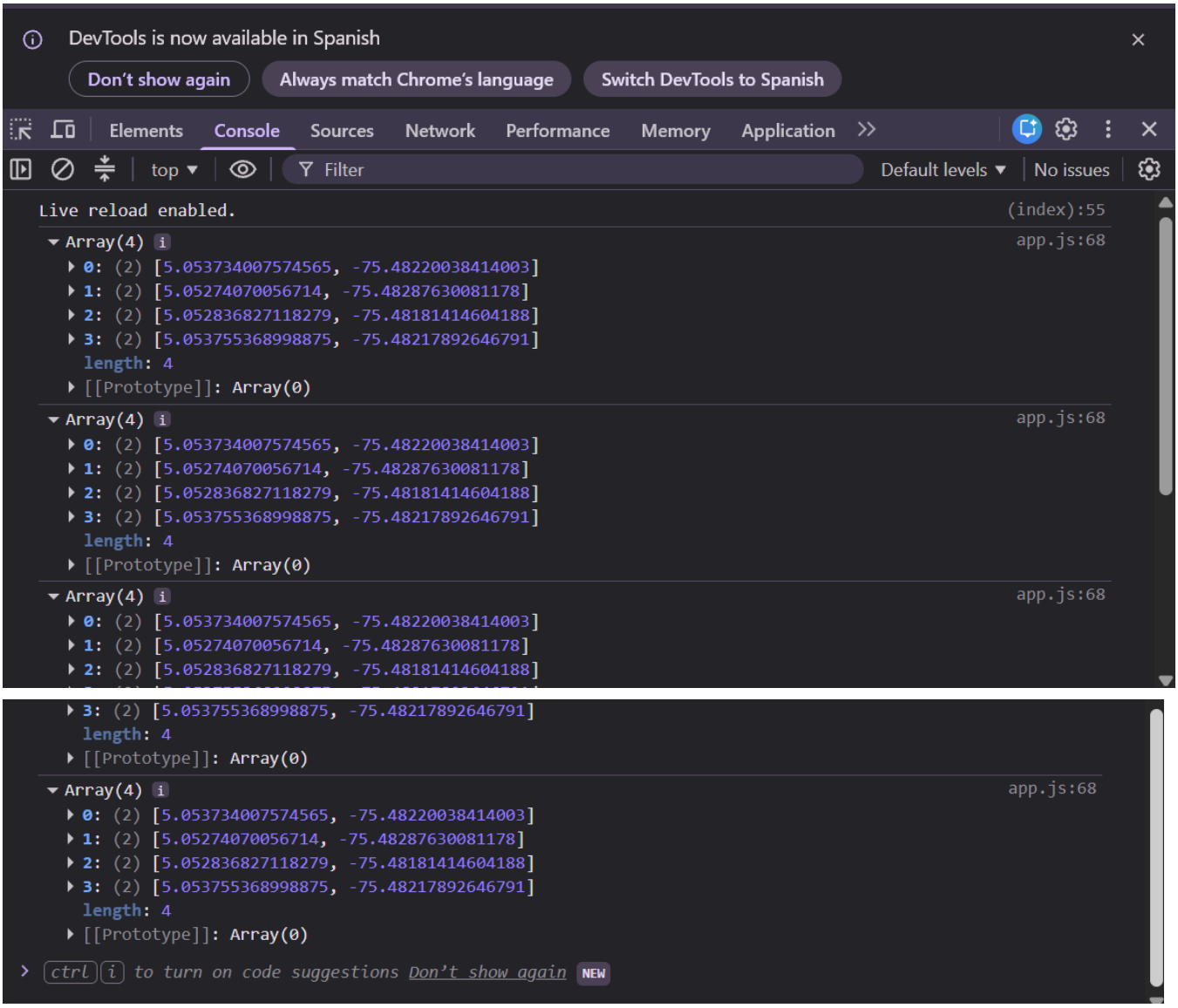
const { lat, lng } = e.latlng;
  puntos.push([lat, lng]);

const v = L.circleMarker([lat, lng], { radius: 6, color: '#e67e22', fillOpacity: 1
}).addTo(mapa);
  vertices.push(v);

lineaGuia.setLatLngs(puntos);
  actualizarBotones();
mostrarInfo(`${puntos.length} vértice(s). Necesitas al menos 3 para cerrar.`);
});

function actualizarBotones() {
  document.getElementById('btnCerrar').disabled = puntos.length < 3 || poligono !==
null;
document.getElementById('btnDeshacer').disabled = puntos.length === 0 || poligono
!== null;
document.getElementById('btnExportar').disabled = poligono === null;
}
```

Verificación: cada clic deja un punto naranja unido al anterior por una línea punteada. Agregue `console.log(puntos)` y revise el array en la consola.



Paso 5. Cerrar el polígono y medirlo (25 min)

Leaflet trabaja con `[lat, lng]`; Turf.js trabaja con GeoJSON `[lng, lat]`. La función `aGeoJSON` hace esa conversión y repite el primer punto al final.

```
function aGeoJSON() {
const anillo = puntos.map(([lat, lng]) => [lng, lat]);
  anillo.push(anillo[0]); // cerrar el anillo
  return turf.polygon([anillo], { nombre: 'Mi polígono', creado: new
Date().toISOString() });
}

function medir() {
const geo = aGeoJSON();
const area = turf.area(geo); // metros cuadrados
```

```
const perimetro = turf.length(turf.polygonToLine(geo), { units: 'meters' }); return
  { area, perimetro };
}

function cerrarPoligono() {
poligono = L.polygon(puntos, { color: '#27ae60', fillOpacity: 0.35 }).addTo(mapa);
  lineaGuia.setLatLngs([]);
mapa.fitBounds(poligono.getBounds());

const { area, perimetro } = medir(); poligono.bindPopup(`Área:
  ${area.toFixed(1)} m²<br>Perímetro:
${perimetro.toFixed(1)} m`).openPopup();
  mostrarInfo(`Polígono cerrado. Área: ${area.toFixed(1)} m² (${(area /
10000).toFixed(3)} ha).`);
actualizarBotones();
}

document.getElementById('btnCerrar').addEventListener('click', cerrarPoligono);
```

Verificación: dibuje una cancha conocida del campus y compare el área calculada con sus medidas reales.

Cancha sintética



Medidas reales:

38 metros de largo por 18 metros de ancho.

Paso 6. Deshacer y limpiar (15 min)

```
function deshacerPunto() {
  puntos.pop();
  mapa.removeLayer(vertices.pop());
  lineaGuia.setLatLngs(puntos);
  actualizarBotones();
mostrarInfo(`${puntos.length} vértice(s).`);
}

function limpiar() {
  puntos = [];
vertices.forEach((v) => mapa.removeLayer(v)); vertices
  = [];
lineaGuia.setLatLngs([]); if
  (poligono) {
mapa.removeLayer(poligono); poligono =
    null;
  }
actualizarBotones();
mostrarInfo('Mapa limpio. Haz clic para empezar de nuevo.');
}

document.getElementById('btnDeshacer').addEventListener('click', deshacerPunto);
document.getElementById('btnLimpiar').addEventListener('click', limpiar);
```

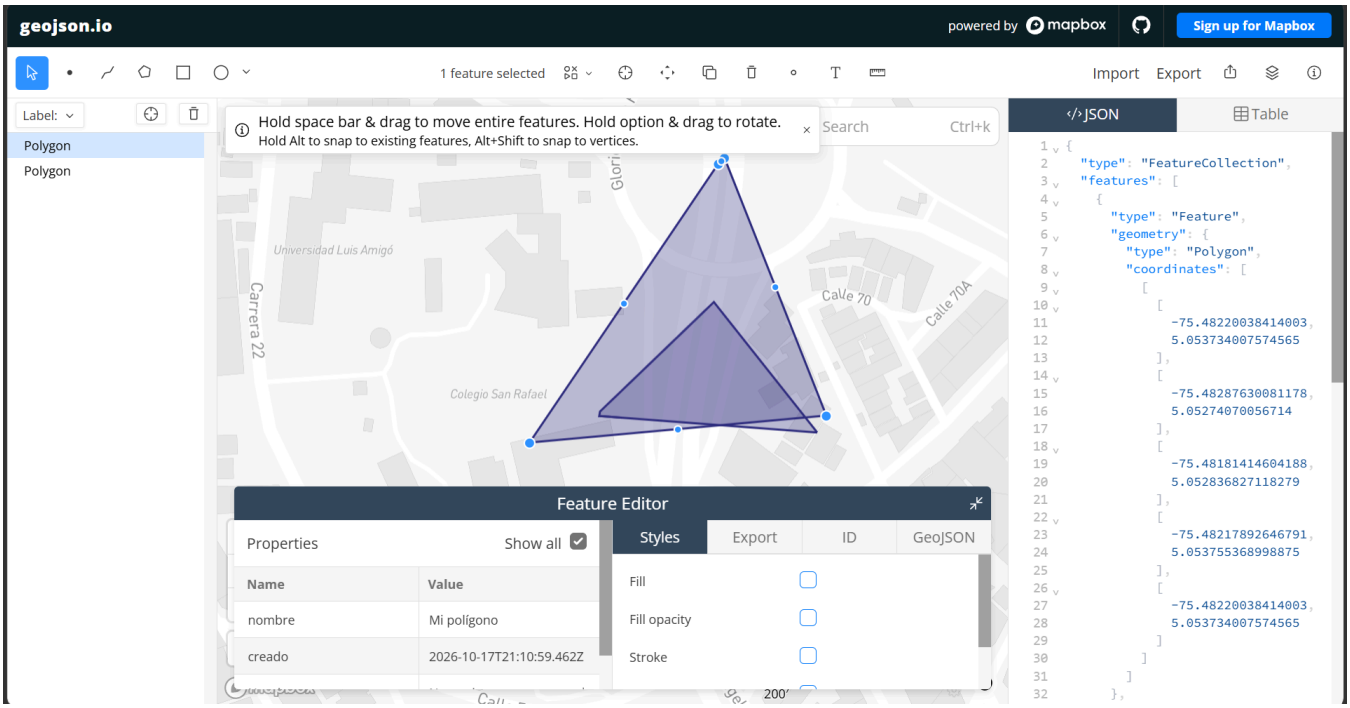
Paso 7. Exportar a GeoJSON (20 min)

```
function exportar() {
const texto = JSON.stringify(aGeoJSON(), null, 2);
const blob = new Blob([texto], { type: 'application/geo+json' });
  const enlace = document.createElement('a');
enlace.href = URL.createObjectURL(blob);
  enlace.download = 'poligono.geojson';
```

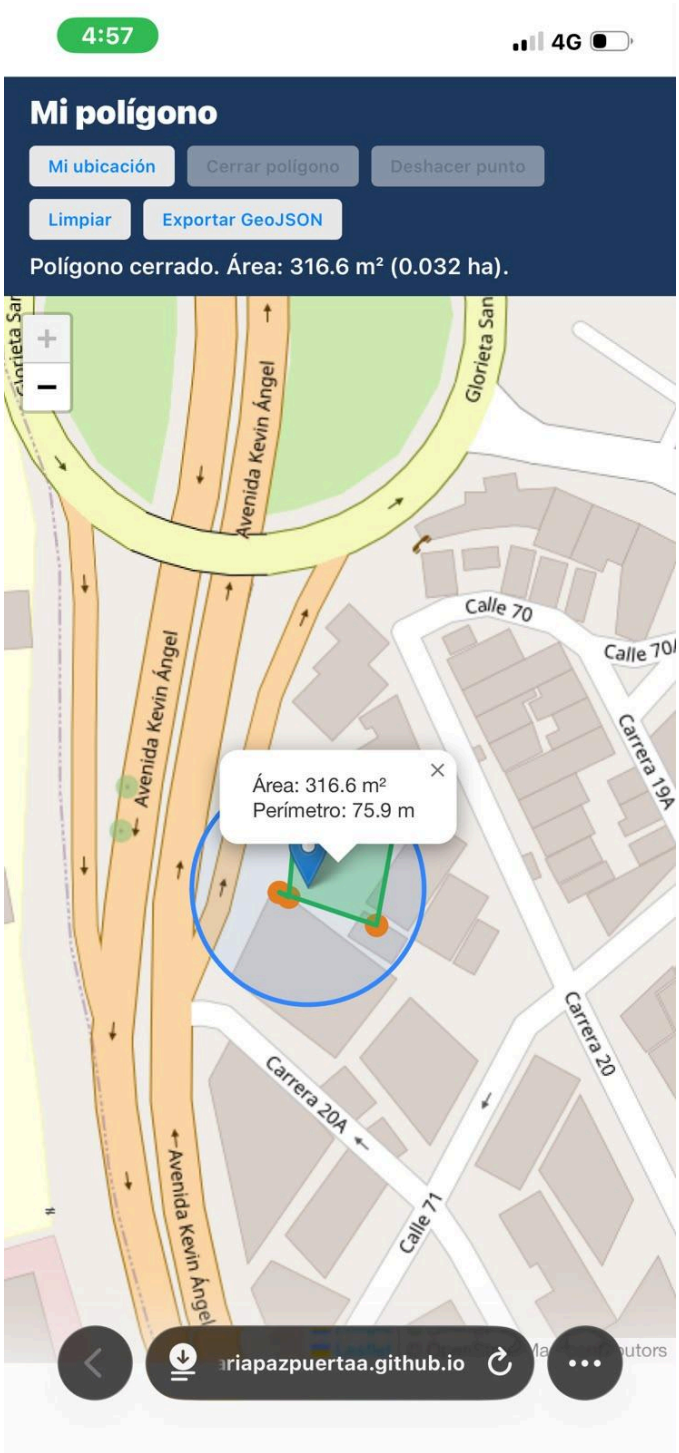
```
enlace.click();
setTimeout(() => URL.revokeObjectURL(enlace.href), 1000);
}

document.getElementById('btnExportar').addEventListener('click', exportar);
```

Verificación: abra geojson.io y arrastre el archivo descargado; el polígono debe aparecer en el mismo lugar.



Nota para probar en celular: la geolocalización exige HTTPS. Live Server en la IP de la red local (`http://192.168...`) no sirve; publique el proyecto en GitHub Pages o Netlify para probarlo con el GPS del teléfono.



Retos para los estudiantes

Cada estudiante resuelve al menos 3 retos; los marcados como avanzados suman puntos extra.

#	Reto	Nivel	Pista
1	Mostrar en una tabla HTML las coordenadas de cada vértice, con 6 decimales	Básico	Recorrer `puntos` con `forEach` y crear filas con `createElement`
2	Permitir elegir el color del polígono con un `<input type="color">`	Básico	`poligono.setStyle({ color })` en el evento `input`
3	Pedir un nombre para el polígono y guardarlo en las propiedades del GeoJSON	Básico	`prompt()` o un `<input>`; propiedad `nombre` en `aGeoJSON`
4	Agregar un vértice con la ubicación actual del usuario (caminar el contorno de la cancha con el celular)	Intermedio	Reutilizar `getCurrentPosition` y hacer `push` a `puntos`
5	Guardar el polígono en `localStorage` y recuperarlo al recargar la página	Intermedio	`JSON.stringify` / `JSON.parse` del array `puntos`
6	Importar un archivo `.geojson` y dibujarlo en el mapa	Intermedio	`<input type="file">`, `FileReader` y `L.geoJSON(datos).addTo(mapa)`

#	Reto	Nivel	Pista
7	Detectar si la ubicación del usuario está dentro o fuera del polígono	Avanzado	`turf.booleanPointIn Polygon(turf.point( [lng, lat]), aGeoJSON())`

8	Rastrear al usuario en tiempo real y avisar cuando salga del polígono (geocerca)	Avanzado	`navigator.geolocation.watchPosition` y `clearWatch`
9	Advertir si el polígono se cruza consigo mismo, porque el área sale mal	Avanzado	`turf.kinks(geo).features.length > 0`

Pregunta de reflexión para cerrar: ¿Qué riesgos de privacidad tiene una página que pide y guarda la ubicación de sus usuarios, y cómo los mitigaría? **(LA PREGUNTA SE RESPONDIÓ EN LA PARTE DE LOS ENTREGABLES).**

Entregables y rúbrica

La nota se calcula sobre 5.0: 4.0 por la aplicación base y 1.0 por los retos.

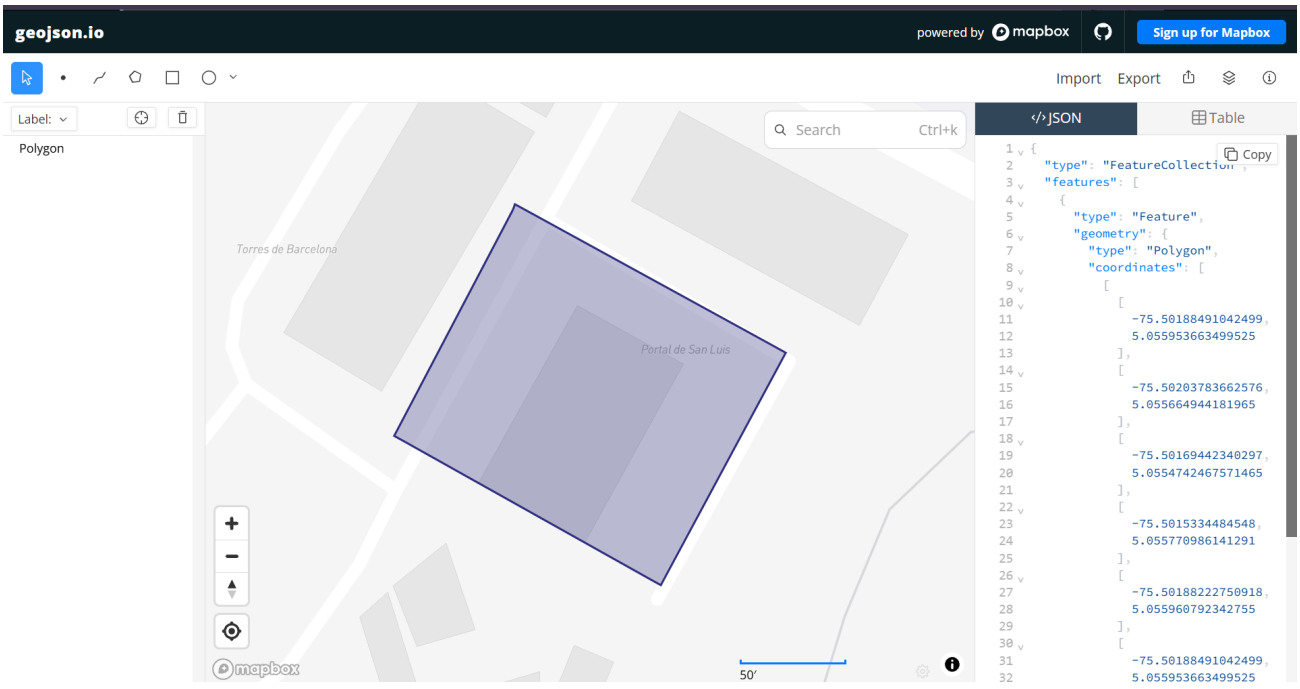
Entregables:

- ☐ **Enlace al repositorio con `index.html`, `estilos.css` y `app.js`**

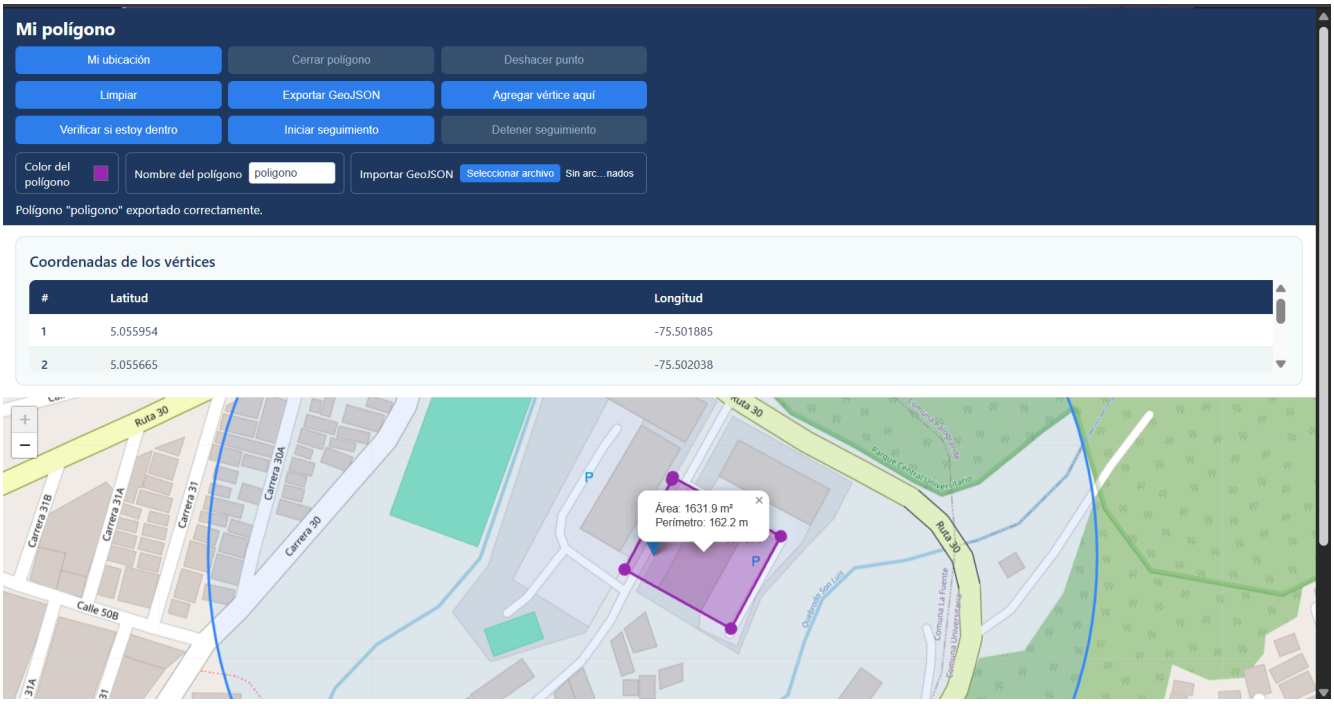
  Enlace al repositorio de Git: https://github.com/MariaPazPuertaa/taller-poligono.git

- ☐ **Enlace a la página publicada en GitHub Pages o Netlify (HTTPS)**
  Enlace de GitHub Pages: https://mariapazpuertaa.github.io/taller-poligono/

- ☐ **Archivo `poligono.geojson` de un lugar real del campus o de su barrio.**



- ☐ **Captura de pantalla del polígono con el popup de área y perímetro.**



- ☐ **Un párrafo con la respuesta a la pregunta de privacidad.**

¿Qué riesgos de privacidad tiene una página que pide y guarda la ubicación de sus usuarios, y cómo los mitigaría?

Los riesgos de privacidad que tiene una página que pide y guarda la ubicación de sus usuarios son muy altos o peligrosos porque cuando se tiene acceso a estos datos tan sensibles, la información almacenada en la página puede ser robada, personas malintencionadas pueden rastrear la ubicación de otras personas e incluso llegar a cometer delitos, como el robo, violar un principio de ética que es guardar la información sin autorización, entre otros. Para mitigar esos riesgos, lo que yo haría primero, sería siempre pedirle la autorización al usuario para acceder a su ubicación, que él pueda escoger que tipo de información o ubicación quiere que sea guardada y por cuánto tiempo quiere que se tenga esta información o ubicación, evitar compartir la información con terceros y por último, utilizar conexiones HTTPS, que consideraría que son las más seguras en comparación con otras que hay (aunque no garantiza seguridad al 100%), para proteger la información guardada, entre otras. Aún así, con todas las opciones de mitigación que propuse, considero que no alcanzaría a cuidar o proteger la información o ubicación del usuario completamente porque hoy en día existen muchas herramientas para robarle la información a las personas o incluso obtener su ubicación para hacer un daño o por cualquier circunstancia. En conclusión, considero que los riesgos al compartir la ubicación son muy peligrosos y por precaución no se deberían de proporcionar a ninguna página, pero hoy en día es una necesidad, sin embargo, si se debería investigar primero si el sitio es seguro y conceder los permisos estrictamente necesarios.

Criterio	Peso	Excelente (100 %)	Aceptable (60 %)	Insuficiente (0 %)
Geolocalización	0.8	Ubica al usuario y maneja los 3 códigos de error	Ubica, pero sin manejo de errores	No funciona
Dibujo del polígono	1.0	Vértices, línea guía y cierre funcionan; no permite cerrar con menos de 3 puntos	Dibuja, pero con fallos de estado	No dibuja
Medición	0.8	Área y perímetro correctos (error menor al 5 % frente a la medida	Calcula, pero con unidades o conversión	No mide

Criterio	Peso	Excelente (100 %)	Aceptable (60 %)	Insuficiente (0 %)
		real)	erradas	
Deshacer, limpiar y exportar	0.8	Las tres funciones operan y el GeoJSON abre bien en geojson.io	Una función falla	Dos o más fallan
Calidad del código	0.6	Nombres claros, funciones cortas, sin código repetido, comentado	Funciona pero desordenado	Ilegible o copiado sin entender
Retos	1.0	9 retos o más, incluido uno avanzado	6 retos	1 o ninguno